

M09 - ETL ELT Data Quality Transformation Engineering

LEARNER BOOK - BEGINNER-FIRST TEACHING RC2

This is the primary teacher.

Do not use the quick reference as the main lesson. Read the concept, run/inspect the demonstration, prove the result, then practise independently.

Contents

DE09-01 - ETL vs ELT and transformation boundaries

DE09-02 - Raw, staging, clean/intermediate and serving contracts

DE09-03 - Deterministic transformations and production SQL style

DE09-04 - Deduplication and key integrity

DE09-05 - Schema checks and data contracts

DE09-06 - Source-to-target reconciliation

DE09-07 - Incremental processing and safe reruns

DE09-08 - Failure behavior, retries, quarantine and atomic state

DE09-09 - Lineage, run metadata and auditability

DE09-10 - Build and defend a reliable batch pipeline

DE09-11 - dbt-style project anatomy, sources and dependency graph

DE09-12 - Executable data tests, quality severity and CI boundaries

DE09-13 - Incremental models, materialization, documentation and handoff

DE09-01 - ETL vs ELT and transformation boundaries

What you will be able to do

Choose ETL/ELT transformation placement based on source/target capability, governance and reproducibility rather than memorizing acronyms.

Why this matters in real Data Engineering

Transformation is the stage where raw meaning becomes business-ready data. Where it runs affects raw preservation, compute cost, security and recovery.

Picture it this way

ETL is preparing ingredients before putting them in the warehouse kitchen; ELT lands ingredients first and cooks inside the powerful warehouse.

Start from zero

- ETL = Extract -> Transform -> Load; ELT = Extract -> Load -> Transform.
- Modern systems often use mixtures: parse/decrypt/minimal validation before load, richer SQL transformations after raw/staging load.
- Preserve raw/landing so transformation logic can be rerun when rules change.
- Push compute where it is governed/scalable, but avoid moving sensitive/raw data into places it is not allowed.
- Transformation boundary should be explicit: which layer owns typing, dedup, business rules and aggregations.
- The acronym is less important than reproducibility, state and contracts.

Teacher demonstration

```
# ETL-style
extract -> validate/transform in Python -> load clean table
# ELT-style
extract -> load raw/staging table -> transform with SQL/models -> serving
```

Read the example like English

- Both can be correct for the same organization in different pipelines.
- In M09 the supplied batch project demonstrates deterministic transformation and an ELT SQL variant.
- Your task is to defend where each rule belongs.

Expected check

decision	question
raw preservation	can logic be replayed?
compute	where is scalable/cheap?
security	where may sensitive raw live?
governance	where are rules/versioning strongest?

Common beginner traps

- Saying ELT is always modern/better.
- Transforming away raw evidence before durable landing.
- Using ETL/ELT names without describing actual layer contracts.

Now you do a similar task

Map the supplied stage09 pipeline as ETL, ELT or hybrid. For five rules (parse type, dedup, join, aggregation, PII masking) state where you would run each and why.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What matters more than the label ETL vs ELT when you are debugging a failed transformation?

Answer in your own words before continuing.

DE09-02 - Raw, staging, clean/intermediate and serving contracts

What you will be able to do

Define raw/staging/clean/intermediate/serving layer contracts so every table/file has a clear grain and allowed transformations.

Why this matters in real Data Engineering

Without layer contracts teams rewrite the same rules in many places and cannot tell which dataset is trustworthy.

Picture it this way

Layers are workstations in a factory: receiving, inspection, assembly, intermediate subassembly and finished product each have different allowed states.

Start from zero

- Raw/landing preserves source representation + ingestion metadata.
- Staging is close to source but parsed/typed enough for transformation.
- Clean/intermediate applies reusable quality/business normalization at declared grain.
- Serving/mart is shaped for consumers and may aggregate/denormalize.
- Each layer contract should state grain, keys, schema, allowed nulls, freshness and owner/source.
- Do not promote records simply because code ran; validation gates move data between contracts.

Teacher demonstration

```
raw_orders      # source-like + run metadata
stg_orders      # typed columns, still source grain
int_order_lines # deduped/joined reusable line grain
mart_daily_sales # one row per date x region
```

Read the example like English

- The names encode intended responsibility, not only storage location.
- A consumer should know whether duplicates/rejects are allowed in each layer.
- Grain normally changes at explicit transformation steps.

Expected check

layer	example grain
raw	one source record
staging	one parsed source record
intermediate	one valid order line
serving	one date-region aggregate

Common beginner traps

- Calling a table clean without a measurable contract.
- Doing business aggregation in raw layer.
- Changing grain without documenting it.

Now you do a similar task

For the supplied project, write a one-sentence contract for raw, staging, clean/intermediate and serving outputs: grain, key, quality expectation and rebuild source.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why is "clean" meaningless unless you define what rules and grain make it clean?

Answer in your own words before continuing.

DE09-03 - Deterministic transformations and production SQL style

What you will be able to do

Write transformations whose result depends only on declared inputs/rules, with explicit SQL columns/joins/filters and stable business logic.

Why this matters in real Data Engineering

A deterministic pipeline is reproducible: same input + code/config yields the same result. Hidden current-time/random/order dependencies make debugging and reruns unreliable.

Picture it this way

A recipe should produce the same cake from the same ingredients; if it secretly depends on which spoon is closest, it is not reproducible.

Start from zero

- Avoid SELECT * in production transformation contracts; list columns so upstream additions do not silently change output.
- Qualify join columns/table aliases and know join cardinality.
- Use explicit CASE/COALESCE rules with documented null semantics.
- Do not use unordered LIMIT to select a "latest" row. Use deterministic ORDER BY with tie-breaker.
- Inject run date/cutoff as a parameter instead of calling current_timestamp throughout historical transformations when reproducibility matters.
- Keep transformations side-effect free until the final atomic materialization/load boundary where possible.

Teacher demonstration

```
SELECT
  order_id,
  customer_id,
  CAST(amount AS numeric(12,2)) AS amount,
  order_date
FROM staging.orders
WHERE order_date >= :window_start
      AND order_date < :window_end;
```

Read the example like English

- Column list creates an explicit output contract.
- Window bounds are parameters, so historical rerun uses the same cutoff.
- Half-open windows reduce overlapping boundary ambiguity.

Expected check

property	evidence
same input/run params	same row/key totals
column contract	explicit list
window	parameterized
ordering when selecting one row	stable tie-breaker

Common beginner traps

- SELECT * as durable model interface.
- current_timestamp changing result on replay.
- Choosing one duplicate with LIMIT 1 and no deterministic order.

Now you do a similar task

Take one transformation in the stage09 TODO/reference and identify every input, parameter and possible nondeterministic dependency. Repair one if present.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What additional information besides the input rows must be fixed to reproduce a time-windowed transformation?

Answer in your own words before continuing.

DE09-04 - Deduplication and key integrity

What you will be able to do

Detect duplicate business keys and apply a documented winner/rejection policy without hiding data loss.

Why this matters in real Data Engineering

Duplicates can be exact retries, legitimate multiple events or conflicting versions. `drop_duplicates()` is not a business rule.

Picture it this way

Two forms with the same ID arrive: you need a rule to decide whether they are identical copies, newer versions or an integrity violation.

Start from zero

- First define target grain/business key.
- Classify duplicate groups: exact duplicates, same key/different payload, legitimate repeated key at a finer grain.
- A winner policy may use source version, `updated_at` plus tie-breaker, or source priority; record why.
- If no trustworthy winner exists, quarantine/conflict rather than choosing arbitrarily.
- Save duplicate counts/keys and reconcile before/after rows.
- Enforce downstream uniqueness with database/test constraints so the problem cannot silently return.

Teacher demonstration

```
-- Example deterministic latest-version rule
ROW_NUMBER() OVER (
  PARTITION BY order_id
  ORDER BY updated_at DESC, source_sequence DESC
) AS rn
-- Keep rn=1 only if that rule matches the contract.
```

Read the example like English

- Partition identifies duplicate business-key group.
- Ordering establishes a deterministic winner.
- Tie-breaker prevents two same-timestamp rows from random winner selection.
- The rejected/older rows still belong in audit/reject evidence if needed.

Expected check

proof	expected
duplicate groups	count + sample keys
policy	written winner/reject rule
post-clean key uniqueness	PASS
row accounting	input = kept + removed/quarantined

Common beginner traps

- drop_duplicates default order.
- Treating repeated customer_id as duplicate when grain is order.
- Deleting losers with no evidence.

Now you do a similar task

Run the duplicate practice batch. Classify duplicate groups, apply a deterministic policy and save pre/post counts + key-uniqueness proof.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why is "keep last" not deterministic unless you define what last means?

Answer in your own words before continuing.

DE09-05 - Schema checks and data contracts

What you will be able to do

Treat schema and data expectations as contracts that fail loudly or quarantine safely when upstream changes.

Why this matters in real Data Engineering

Transformations are built on assumptions. A renamed/missing/type-changed column can create wrong output even when parsing succeeds.

Picture it this way

A contract is a plug specification: if a supplier changes connector shape/voltage, the machine should stop safely rather than force it in.

Start from zero

- Check required columns and unexpected/optional policy before transformation.
- Check parseable types/ranges/nullability and accepted categorical values where appropriate.
- Schema drift can be breaking, compatible or ignored only by explicit rule.
- Capture observed schema + contract version in run metadata.
- Fail batch when interpretation is unsafe; quarantine only rows when record-local defects can be isolated.
- Version contracts along with code and coordinate producer/consumer change.

Teacher demonstration

```
REQUIRED = {"order_id", "customer_id", "amount", "updated_at"}
missing = REQUIRED - set(df.columns)
if missing:
    raise ValueError(f"breaking schema: {sorted(missing)}")
```

Read the example like English

- The check fails before business transformation consumes an incomplete schema.
- You can separately define optional/allowed-extra columns.
- The error should surface the contract problem, not be converted to all-null output.

Expected check

change	policy example
new optional column	allow + record
missing order_id	batch fail
amount text garbage	record reject or batch threshold
renamed field	breaking until contract update

Common beginner traps

- Auto-creating missing critical columns as null.
- Accepting every extra/type change silently.
- Schema check only after target was already overwritten.

Now you do a similar task

Run the schema-drift batch through your pipeline. Save observed vs expected schema and prove the prior serving output/state remains safe on failure.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

When should schema drift reject one row versus stop the whole batch?

Answer in your own words before continuing.

DE09-06 - Source-to-target reconciliation

What you will be able to do

Reconcile source, rejected, transformed and target counts/amounts at meaningful grains so silent loss/duplication is detectable.

Why this matters in real Data Engineering

A pipeline can run without exceptions and still lose rows or multiply money through a join. Reconciliation is correctness evidence.

Picture it this way

Accounting books must balance: money/records entering a controlled transformation should be explainable on the other side.

Start from zero

- Use row-count reconciliation only when grain/filter semantics make it meaningful.
- Add key uniqueness, distinct business-key and numeric control totals to catch fan-out/loss.
- Reconcile before/after joins at the appropriate grain.
- Track explicitly excluded/cancelled/rejected rows instead of pretending they vanished.
- Compare source-to-target over the same time/window, not mismatched populations.
- Treat unexplained mismatch as failure, not a warning to ignore.

Teacher demonstration

```
assert source_rows == valid_rows + rejected_rows
assert len(valid_df) == valid_df["order_id"].nunique()
assert round(valid_df["amount"].sum(),2) == expected_valid_amount
```

Read the example like English

- First invariant accounts every source row.
- Second protects target business-key grain.
- Numeric total catches errors that row count cannot.
- Expected totals must refer to the same filtered population.

Expected check

control	catches
row count	loss/extra rows
distinct keys	duplicates/fan-out
amount sum	measure corruption
reject reasons	explained exclusions

Common beginner traps

- Comparing counts at different grains.
- Ignoring a small monetary mismatch.
- Valid+rejected does not equal source but job still marked success.

Now you do a similar task

Run the reconciliation lab on the supplied batches. Save at least row, distinct-key and amount controls for one successful run and one failed/quality run.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

How can source row count equal target row count while the transformation is still wrong?

Answer in your own words before continuing.

DE09-07 - Incremental processing and safe reruns

What you will be able to do

Process only new/changed input while preserving deterministic reruns and preventing target duplication.

Why this matters in real Data Engineering

Incremental pipelines are stateful. Correctness depends on what is already processed and how the same window behaves on retry.

Picture it this way

Incremental processing is adding new pages to a ledger: if you reopen yesterday, you need a rule that does not write the same transaction twice.

Start from zero

- Incremental boundary may be source file identity, watermark, partition/date or CDC position.
- Persist committed state only after target/update succeeds.
- Target write should use unique key/upsert/partition replacement or other idempotent strategy.
- Rerunning the exact same batch should not double business rows.
- Late updates/backfills need explicit affected-window handling.
- Record which input identities/state produced each target partition/model.

Teacher demonstration

```
# Conceptual safe sequence
previous_state = load_state()
rows = extract_after(previous_state)
result = transform(rows)
write_target_idempotently(result)
validate_target()
save_new_state() # last
```

Read the example like English

- State save is deliberately after validated durable target write.
- If failure occurs before save, replay may repeat work; idempotent target makes that safe.
- If state were saved first, failure could skip unfinished rows forever.

Expected check

test	expected
first incremental run	new rows added/updated
same batch rerun	no duplicate business keys
failure before state commit	old state retained
late/backfill	explicit window/policy

Common beginner traps

- Appending every incremental batch without key/partition strategy.
- Saving state before target commit.
- Using filename timestamp as watermark without source contract.

Now you do a similar task

Run batch_001 then batch_002_incremental, record state/target counts, rerun batch_002 and prove the target does not double.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why is idempotency essential when state is committed after the target write?

Answer in your own words before continuing.

DE09-08 - Failure behavior, retries, quarantine and atomic state

What you will be able to do

Design failure behavior that preserves prior good serving data/state, retries transient operations and quarantines deterministic bad input.

Why this matters in real Data Engineering

A robust batch pipeline is defined by what happens when it fails halfway, not only by the happy path.

Picture it this way

If assembly fails, do not throw away yesterday's good product or stamp the new batch as shipped. Keep failed parts isolated and the official inventory unchanged.

Start from zero

- Classify failures: transient infrastructure vs deterministic data/code/contract.
- Retry bounded transient steps only when safe.
- Quarantine bad records/files with reason and run identity.
- Write new output to temporary/staging location/table before atomic swap/commit when possible.
- Advance state only after successful validated publish.
- On failure, retain prior serving artifact and state, and make run status FAILED/INCOMPLETE visible.

Teacher demonstration

```
# publish pattern idea
build temporary_target
validate temporary_target
BEGIN/atomic rename-or-transaction
  replace/publish serving target
  commit state
END
# If validation fails, serving remains previous good version.
```

Read the example like English

- Atomicity means consumers do not see a half-written serving dataset.
- Quarantine keeps evidence but should not magically convert a breaking batch into success.
- Retries belong around transient operations, not around a failing deterministic schema test.

Expected check

failure	safe behavior
bad schema	fail/quarantine input; no publish/state advance
temporary DB/network	bounded retry
validation mismatch	no serving swap
success	publish + state/run commit

Common beginner traps

- Overwriting serving table/file before validation.
- Retrying schema/data-quality failure endlessly.
- Advancing state on failed publish.

Now you do a similar task

Run the bad-quality/schema-drift failure exercise. Prove previous serving output/state remains unchanged and show failure/quarantine evidence.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What does "atomic publish" protect consumers from?

Answer in your own words before continuing.

DE09-09 - Lineage, run metadata and auditability

What you will be able to do

Capture lineage and run metadata connecting source identities, code/config version, transformations and target outputs.

Why this matters in real Data Engineering

When a number is questioned, you need to answer which input/run/code created it and how to reproduce it.

Picture it this way

Lineage is a parcel tracking chain from supplier box through every workstation to the finished shelf item.

Start from zero

- Record run_id, start/end/status and source input identities/hashes/windows.
- Record code version/commit/config/contract version when practical.
- Record output target/partition/version and control totals.
- Column-level lineage is useful for critical derived fields, but start with truthful dataset/run lineage.
- Audit metadata must be immutable enough to investigate past runs.
- Do not put passwords/tokens/PII payloads into lineage logs.

Teacher demonstration

```
run = {
  "run_id": "20260828T010000Z-b001",
  "input": "batch_001.csv",
  "input_sha256": "...",
  "code_version": "git:abc123",
  "status": "SUCCESS",
  "source_rows": 10,
  "target_rows": 9,
  "rejected_rows": 1
}
```

Read the example like English

- Run metadata explains the one-row difference.
- Input hash distinguishes content, not just filename.
- Code version makes historical reproduction possible after logic changes.

Expected check

question	metadata needed
what input?	path/object + hash/window
what code?	commit/version
what happened?	status/counts/rejects
where output?	table/file/partition/version

Common beginner traps

- Logging only start/end time.
- Lineage file can be edited silently with no run identity.
- Recording secrets to make debugging "easier".

Now you do a similar task

Add/inspect lineage metadata for one stage09 run. Starting from a target row, write how you would identify source batch and code version.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

If a filename is unchanged but its contents were replaced, what lineage field helps expose that?

Answer in your own words before continuing.

DE09-10 - Build and defend a reliable batch pipeline

What you will be able to do

Assemble a reliable batch transformation with explicit contracts, deterministic logic, tests, reconciliation, safe state and failure recovery.

Why this matters in real Data Engineering

The pipeline practical is where individual quality concepts become one correctness argument.

Picture it this way

You are operating a production line: every batch has receiving evidence, tested transformations, balancing controls, a publish gate and a recoverable state receipt.

Start from zero

- Start by documenting input/window and layer contracts.
- Run schema/quality checks before unsafe transformations.
- Apply deterministic dedup and transformation rules at declared grain.
- Reconcile every boundary and target control total.
- Publish atomically/idempotently and commit state last.
- Save run lineage/status so a failed or replayed run is explainable.
- Prove with tests and a second rerun, not only by reading code.

Teacher demonstration

```
# Practical invariants
source_rows == valid_rows + rejected_rows
serving_business_key_unique == True
source_valid_amount == serving_amount    # same population/grain
failed_run_does_not_change_good_serving == True
same_incremental_rerun_does_not_double == True
```

Read the example like English

- These invariants form a defense of correctness.
- Use the supplied TODO first; keep reference closed until after genuine attempt/repair.
- The practical should include a bad batch and replay, not only happy-path batch.

Expected check

area	evidence
contract	schema/grain/key
quality	rejects/tests
reconciliation	rows/keys/measures
state	before/after + failure behavior
lineage	run/input/code/output

Common beginner traps

- Finishing only the happy path.
- Opening pipeline_reference.py before an independent attempt.
- Calling a target reliable because unit tests pass but row controls do not.

Now you do a similar task

Complete the supplied Stage 09 pipeline TODO closed-reference, run tests, run good/incremental/bad scenarios and defend every count/state decision.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Which single pipeline property would you test first on a rerun, and why?

Answer in your own words before continuing.

DE09-11 - dbt-style project anatomy, sources and dependency graph

What you will be able to do

Understand a dbt-style analytics engineering project as a directed dependency graph of sources, models, tests and documentation.

Why this matters in real Data Engineering

As SQL transformations grow, file names and manual run order become brittle. Model graphs make dependencies explicit and executable.

Picture it this way

A dbt-style project is a recipe book where each recipe declares which earlier dish/source it depends on, letting the system build the dependency graph.

Start from zero

- A **source** declares an upstream relation owned outside the transformation project.
- A model is usually a SELECT statement materialized as view/table/incremental according to config.
- **ref()**-style dependency references create graph edges rather than hardcoded environment table names.
- The DAG determines build/test order from dependencies.
- Staging models normalize source names/types; intermediate models combine reusable logic; marts serve business questions.
- Do not hide critical transformation steps in ad-hoc manual SQL outside the versioned project.

Teacher demonstration

```
# Conceptual dbt-style graph
source('app', 'orders')
  -> stg_orders
  -> int_valid_orders
  -> fct_orders
source('app', 'customers')
  -> stg_customers
  -> int_valid_orders
```

Read the example like English

- Two sources feed a shared intermediate model.
- The final fact depends transitively on all upstream nodes.
- A change/test can be targeted based on graph descendants/parents in mature tools.

Expected check

node type	role
source	external declared input
staging	light source normalization
intermediate	reusable transformation
mart/fact/dim	consumer-ready model
test/docs	contract/evidence

Common beginner traps

- Hardcoding production database names in every model.
- One giant SQL file with no reusable/testable boundaries.
- Using dbt terminology but not declaring grain/keys/tests.

Now you do a similar task

Take the stage09 transformations and sketch a dbt-style DAG with source, staging, intermediate and serving nodes. State grain/key for each model.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What does a dependency graph give you beyond putting SQL files in numbered folders?

Answer in your own words before continuing.

DE09-12 - Executable data tests, quality severity and CI boundaries

What you will be able to do

Turn quality expectations into executable tests with severity and a CI boundary that blocks unsafe changes.

Why this matters in real Data Engineering

A documented rule that nobody runs is weaker than a test that fails the build/publish when the contract is broken.

Picture it this way

Executable data tests are automated inspectors on the factory line. Severity decides whether a defect stops shipment or raises a warning for follow-up.

Start from zero

- Common tests: not-null, uniqueness, accepted values, relationships, row-count/control-total and custom business assertions.
- Test at the layer where the invariant should hold; raw data may intentionally contain defects that clean/serving cannot.
- Classify severity: error/blocking vs warning/monitoring.
- CI should run code/unit/static tests and data-model tests against a controlled environment/sample before merge/deploy.
- A test failure should show enough context/key samples to repair without dumping sensitive data.
- Do not lower severity merely to make CI green; change the rule only with an approved contract decision.

Teacher demonstration

```
# Pseudocode tests
assert mart_orders["order_id"].is_unique
assert mart_orders["customer_id"].notna().all()
assert set(mart_orders["status"]) <= {"PAID", "SHIPPED", "CANCELLED"}
assert source_valid_amount == mart_amount
```

Read the example like English

- These tests target different invariants: key, nullability, accepted domain and reconciliation.
- The final reconciliation test can catch problems that individual row tests miss.
- CI becomes a gate when a blocking test changes the exit/build status.

Expected check

test	typical severity
primary business key unique	ERROR
required FK not null	ERROR
freshness slightly late	WARN then escalate by SLA
amount reconciliation	ERROR

Common beginner traps

- Testing only syntax/row count.
- Turning every test into warning.
- Running tests after publishing consumer-visible data.

Now you do a similar task

Write five executable tests for the stage09 serving output and classify each ERROR/WARN with reasoning. Simulate one failure and prove the publish/CI decision changes.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why is a warning-only uniqueness test usually a dangerous design for a fact/model key?

Answer in your own words before continuing.

DE09-13 - Incremental models, materialization, documentation and handoff

What you will be able to do

Choose materialization/incremental strategy, document model contracts and produce a handoff another engineer can rebuild safely.

Why this matters in real Data Engineering

Analytics transformations live for months. Materialization, incremental predicates and documentation determine cost, freshness and rerun behavior.

Picture it this way

A model can be a live recipe (view), pre-cooked table, or table updated only with new ingredients. The choice changes runtime and state responsibility.

Start from zero

- A view stores query logic; a table stores computed results; incremental materialization updates a subset/statefully.
- Incremental models require a unique key, changed-row detection/window and safe rerun/upsert strategy.
- Full refresh is a recovery/change tool; do not assume it is always cheap or always possible.
- Document model purpose, grain, keys, important columns, source dependencies and tests.
- Generate/retain lineage graph and run/test results as handoff evidence.
- A clean rebuild in a fresh schema/environment is the strongest handoff test.

Teacher demonstration

```
# Incremental model design questions
unique_key = "order_id"
change_key = ("updated_at", "order_id")
strategy = "merge/upsert"
late_data_policy = "3-day overlap + key merge"
full_refresh = "supported after raw history validation"
```

Read the example like English

- Each line is a state/correctness decision, not configuration decoration.
- Documentation should match executable behavior/tests.
- Fresh rebuild validates that no hidden manual steps are required.

Expected check

handoff	must explain
materialization	view/table/incremental + why
grain/key	one sentence + unique test
incremental state	predicate/upsert/late policy
rebuild	full-refresh/fresh-schema steps
tests	blocking invariants

Common beginner traps

- Choosing incremental only because it sounds advanced.
- No unique key for merge.
- README says one grain while tests/SQL implement another.

Now you do a similar task

Design a dbt-style handoff for the Stage09 serving model: DAG, materialization, key, incremental policy, tests and fresh rebuild steps.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What makes an incremental model stateful even if its SQL looks like a normal SELECT?

Answer in your own words before continuing.